

Opus 4.8 用户日志验收标准-0630

- 适用范围：Claude Opus 4.8 真实用户交互轨迹采购
- 关键变更 (vs v1) :
 - a. 交付格式从「OpenAI ChatMessage」切换到 **Anthropic 原生 API request / response** (保留 thinking block 的 `signature`、`tool_use` / `tool_result` 原始结构), 必须携带 **signature**
 - b. 交付粒度：一条 jsonl 记录 = 一次 LLM call 的 **raw request / response**, 靠 `session_id` 关联同一会话的多条记录
 - c. `session_id` 仅作为客户端会话生命周期标识 (粗粒度容器), 上游不需要做 trajectory 切分、上下文管理识别、最长前缀去重
- 不再要求：上游做格式转换、合并 trajectory

1. 内容要求

- 真实用户交互 agent 轨迹 (人在客户端 / IDE / 网页里实际发起的对话)
- 禁止：terminal / sandbox 预设任务、batch API 程序化调用、合成数据
- 禁止：cron / heartbeat / no_reply 类机器轮 (识别详见 §4.3)
- 尽量不做数据脱敏, 如果做脱敏需要指出

2. 模型要求

claude opus 4.8, thinking_effort = max, thinking type = adaptive

3. 格式要求 — Anthropic 原生 / Call-level

1. 不进行任何脱敏/篡改上下文操作, 必须是原始日志, 避免前缀合并失败

3.1 数据样例



复制代码

```

{
  "role": "assistant",
  "content": [
    {
      "type": "thinking",
      "thinking": "",
      "signature":
"EoMLCmMIDxABGAIqQNAowq+ZC4r0RnM9aHPjVqjab0oh7Nyu0mAjniSL7UU/tMxhOCS0Er+
dF9B14oQcASfIqZi1P8lDVmmH77LSLVwyDWNsYXVKZS1xdWluY2U4AEIIdGhpbmtpbmcSDGX
RKV5mdt181P5LMhoMNEdkSgK/ienwK+rbIjAwhydtFGhY3Uhj7abC3ycw/Ad0T/6BOCW4zI1
a2smahQMddZF850meh/q5V0sTDlwqzQ1AaHJMTQ4UVm6CR8SJbjj0wX0us1M7aAYKxWTjw6k
iwMRByB7qKUuSioQ+jjRcyBNwuTBZlxaUxxQGbzqpgqurY/Hz1eHRm41IiRau5ROH5oejU3w
cYJyI47hwtBQhTlS8dVxib1bQFfEj92v0aBvTQLikgW6uX7i7bS1QuDBeD4eDgQ1IcApKxTd
yJjfKxx5xj2RQbL2NEY45PDbM/WlbaVLfXx3GnzvYgSs6jlR4LoNvDpQwNzI7UVQh+YnJlh8
t7giXZDLpP5edhAj4GVms8eqvG1sGyBe8ML+NU68onXNtjjv94Yvu9Zro2BeB6m7n0/dHKuz
tf10RSZUizXI5M4kGEety809BNyQ8geq0AQyQK7krJXTb2r3yZ10Vk03KJI6csLHREJqC1Qo
mqyfHCaRuVq1mjaDwxXfG0cUXb/J+keMU05hsTRmuQCB/NDN4lqjBB40qMNAsiE18YzQuqtN
NjS/R2Wg+N+qB2nGERdbpLMIIb5o1mj1Brqphn1WYTU0kARpLxUZkgT9bty2f9nLYz1YWmrv
ICYWz/BEFEpmPeLewLZ8xyEBjOI6WadC+aSDh4X+nhrTKWXlNy3BkkSna+NedieC751PtXPj
ape4U8EDilJ7kwGrGVu/eZmfsV5pN0CpWYR8MpmRMzXAcuAokr6p5+iWYOINu0eUba6npadD
xmDY+0aDC4Vq+a0wTvX9hGV9tWKEBJiY39Jd8q90pB3vMiLy0m9WTQq1Lez1MYHkfXrhNPDI
gAjK4LT6MqIuToJVaGJxZfzLDD8JBvL9hA+8SmXymD6LYMQVD0LXh3XxJno5eRAVP0SeJQND
mm0/qqrHbHdd9F4CA/Gln0VPZd25D0om3XtgQ0S84fyVW0E603oEaa0gvarlg/EmNtck6uMC
JrA/a7PiRdQruif+GxLb9mpUEAwR50TPHZUCANbWu+NY2xzzV/bG1Mr/CYDyc5U6tMkSQr94
ML1rZKRAiyIg0QKdps9Qp0NvWrc/iLSAvLIEErU0tQnxTrv3a6msqwmwxho6PbNZYurPJ7TA
HX2wS1rzV0y4d0s9Pmxukb48hf1FzDAqA606pItJMTQKUDwbknicYDpv1FTFmI1xhOJErLrp
JFGTScjg7arn5NkfipMcfq1qq9xexfxb9VxakxYG/J0X777Y81cndq7FG4UNYYtELiuN7Fdm
lPOk7q+6ldzuR8v10epSrKtJjJgjuZAd4xpJBaZkm/md0Qx3L5mBj3zkZI6Z1LuhZXZFv1zZ
9uJTOZRA67Sxasx8SzVb4CLNvLrXqG33zG555CQtpK/wo/a1eNTPQAB8SGGmjLeABcIjru/r
AuNWroTb4f4HqBQaIVB3jNKHqNK5I1I9VcZjyywRswNpVFoMe4+5oap9X+jZ+94YwLraSIfu
U3+Ls03iY4JEnlu7OWHcWnVtuiS1fmrQsEP/fCI/myJq/B9MJAeg1eNlq+iQKfc6BS0iDmzO
mheDFpdEbGjjzzigorG0nHn3ch81JvraWD+y238giXHSVGLn8mgFjKhnyKiRjXNsCSVuTArV
7mytJmHdPffPQU4WB4A6D+G401kwypxVtyHFPvH4b69MQ7VDqDXMFyEyUZKee8MvfxKaNOX0
9ZPZBfx0ZVrNBGxgB"
    },
    {
      "type": "text",
      "text": "I'll start by exploring the input files to
understand the data format before building the analysis pipeline."
    },
    {
      "type": "tool_use",
      "id": "toolu_bdrk_01TzStRBT8CaKTgFaLXoakWL",
      "name": "Bash",
      "input": {

```

```

        "command": "echo \"=== /workspace exists? ===\" && ls -la
/workspace/ 2>&1; echo \"=== vocab.txt ===\" && ls -la
/workspace/vocab.txt 2>&1; echo \"=== corpus dir ===\" && ls -la
/workspace/corpus/ 2>&1",
        "description": "Explore workspace input files"
    }
}
],
},
{
    "role": "user",
    "content": [
        {
            "type": "tool_result",
            "tool_use_id": "toolu_bdrk_01TzStRBT8CaKTgFaLXoakWL",
            "content": "=== /workspace exists? ===\ntotal 16\ndrwxr-xr-
x 1 root root 4096 Jun 15 02:07 .\ndrwxr-xr-x 1 root root 4096 Jun 27
14:12 ..\ndrwxr-xr-x 2 root root 4096 Jun 15 02:07 corpus\n-rw-rw-r-- 1
root root 159 Jun 15 02:07 vocab.txt\n=== vocab.txt ===\n-rw-rw-r-- 1
root root 159 Jun 15 02:07 /workspace/vocab.txt\n=== corpus dir
===\ntotal 24\ndrwxr-xr-x 2 root root 4096 Jun 15 02:07 .\ndrwxr-xr-x 1
root root 4096 Jun 15 02:07 ..\n-rw-rw-r-- 1 root root 104 Jun 15
02:07 doc1.txt\n-rw-rw-r-- 1 root root 108 Jun 15 02:07 doc2.txt\n-rw-
rw-r-- 1 root root 115 Jun 15 02:07 doc3.txt\n-rw-rw-r-- 1 root root
114 Jun 15 02:07 doc4.txt"
        }
    ]
},
{
    "role": "assistant",
    "content": [
        {
            "type": "thinking",
            "thinking": "",
            "signature":
"EvABCmMIDxABGAiQdDf1xeTysGGNk3uzQ9I6XiKcYv1I0v2QA0EV3LQ4ZEFbeZMWOUoYx1
T2mlXXEB8ZY6nPXaDZT7bbzY1uE33kQgyDWNsYXVkJZS1xdWluY2U4AEIIIdGhpbmtpbmcSDJE
QNGc/5cYFCQ75JhoMin44NNN5QBR1IxmfiJDB0GnSnc0r2PEm5XrMiMRS6EBfSHYRyqxLC/R
tHqM9hr0aI7eZLsh0SwHeKkEXN1IqO92ax2SNcSRU0ptHGKvY00wRLDQv5s2BFILVhraNx5w
x7e0vd702QCfvTyjMtK2pYqk1yd15+jWQj0R5GAE="
        },
        {
            "type": "text",
            "text": "Let me examine the contents of all input files."
        }
    ],

```

```

{
  "type": "tool_use",
  "id": "toolu_bdrk_01FeeiBzzNUBJH1n6mKCpArW",
  "name": "Read",
  "input": {
    "file_path": "/workspace/vocab.txt"
  }
},
{
  "type": "tool_use",
  "id": "toolu_bdrk_013vQNdE6uKTPyneH7iUHMxh",
  "name": "Read",
  "input": {
    "file_path": "/workspace/corpus/doc1.txt"
  }
},
{
  "type": "tool_use",
  "id": "toolu_bdrk_01AnKpWwnJWULTJyitR9UWX8",
  "name": "Read",
  "input": {
    "file_path": "/workspace/corpus/doc2.txt"
  }
},
{
  "type": "tool_use",
  "id": "toolu_bdrk_01Hn4nDMAaME6oRNtNEpSG7D",
  "name": "Read",
  "input": {
    "file_path": "/workspace/corpus/doc3.txt"
  }
},
{
  "type": "tool_use",
  "id": "toolu_bdrk_011XwzrV1YJEUapfF8iYtNGW",
  "name": "Read",
  "input": {
    "file_path": "/workspace/corpus/doc4.txt"
  }
}
]

```

3.2 字段说明

字段	必填	说明
<code>session_id</code>	✓	客户端会话生命周期 ID（粗粒度）。一个 <code>session_id</code> 下可包含多次 LLM call
<code>request_id</code>	✓	单次调用的全局唯一 ID（Anthropic 返回的 <code>x-request-id</code> 或自定义）
<code>timestamp</code>	✓	ISO8601 UTC，用于排序
<code>thinking_effort</code>	✓	<code>high</code> / <code>xhigh</code> / <code>max</code> （取自客户端用户实际选择档位，不是从 <code>budget_tokens</code> 反推）
<code>is_garbled</code>	可选	<code>messages</code> / <code>system</code> / <code>tool_result</code> 含非 UTF-8 / mojibake 时置 true
<code>request</code>	✓	Anthropic Messages API 的 raw request body（直接是 body ，不要套 <code>request.body</code> 这层 wrapper）
<code>request.system</code>	✓	content blocks list，禁止字符串化、禁止把 tool schema 拼进 system text
<code>request.tools</code>	✓	每个 tool 必须有 <code>name</code> / <code>description</code> / <code>input_schema</code> （禁止用 OAI 的 <code>function.parameters</code> 形态）
<code>request.messages</code>	✓	Anthropic 形态： <code>user</code> / <code>assistant</code> ，content 为 block list， 携带signature的比例应该> 65%

与 `claude-trace` 工具产物的差异（防误交）

如果上游用 `claude-trace` 类客户端工具采集，会得到这种形态：



```
// ✗ 不要按这个交！这是 claude-trace 调试工具产物，不是采购格式
{
  "request": { "url": "...", "body": { ... API request body ... } },
  "response": { "status_code": 200, "body": { ... } 或 "body_raw":
"data: ..." }
}
```

需要做以下两步转换才能符合本 spec：

1. 把 `request.body.*` 提升一层 → `request.*`
2. 解码 SSE 流（`response.body_raw`）→ 完整的 Anthropic response body，落到 `response.response_data.*`；解码逻辑参考 `convert_anthropic_trace_to_oai.py::parse_response`

Session 切分约束（粗粒度）

`session_id` 需保证：

- 同一 `session_id` 下的 calls 来自同一客户端 + 同一用户 + 同一上下文对话（不能跨用户/上下文混用）
- 不要求 calls 之间严格连续 —— 即使中途发生 `/clear` / `/compact` / token 自动截断 / 用户手动开新会话，也不必拆 `session_id`
- **session的组织形式**：最好为一个session一个文件夹，文件夹下一个文件为一个request

Block 类型保真要求

Block	必须保留的字段	禁止行为
<code>thinking</code>	<code>type</code> / <code>thinking</code> / <code>signature</code>	✗ 拼成字符串；✗ 丢弃 signature
<code>redacted_thinking</code>	<code>type</code> / <code>data</code>	✗ 整块丢弃
<code>text</code>	<code>type</code> / <code>text</code>	—
<code>tool_use</code>	<code>type</code> / <code>id</code> / <code>name</code> / <code>input</code>	✗ 转写为 OAI <code>tool_calls</code> ；✗ <code>input</code> 字符串化
<code>tool_result</code>	<code>type</code> / <code>tool_use_id</code> / <code>content</code> （出错时含 <code>is_error</code> ）	✗ 转写为 OAI <code>role: tool</code>

image

type / source

✗ 内联 base64 改成 URL

3.3 必须满足 (ERROR — 任一不满足则单条 REJECT)

Session 级

- ✓ `session_id` 跨 session 唯一 / `request_id` 全局唯一
- ✓ `model` ∈ §2 允许集合
- ✓ 同一 `session_id` 下的所有 call 的 `request.model` 一致
- ✓ `thinking_effort` ∈ {high, xhigh, max}

Call 级

- ✓ `request.system` 非空 (禁止把 tools schema 拼到 system 里替代独立 tools 字段)
- ✓ `request.tools` 是 list 且每个 tool 含 `name` / `description` / `input_schema`
- ✓ `request.messages[0].role == "user"`
- ✓ 所有 `assistant.content` 为 block list (不允许是裸字符串)
- ✓ 每个 `thinking` **block 必须** `thinking` **非空且** `signature` **非空**
- ✓ 每个 `tool_use` 必须 `id` / `name` / `input` 全有; `input` 是 dict (不是 JSON 字符串)
- ✓ 每个 `tool_result.tool_use_id` 必须能在该次 call 上文 `tool_use.id` 中找到配对
- ✓ `response.response_data.stop_reason` ∈ {end_turn, tool_use, max_tokens, stop_sequence}, 不能为 null
- ✓ `response.response_data.content` 为 block list

禁止行为

- ✗ 把 thinking blocks 拼接成字符串、丢弃 signature
- ✗ 把 `tool_use` / `tool_result` 转写为 OAI `tool_calls` / `role=tool`
- ✗ 把 `system` content 字符串化或拼到 user message 里
- ✗ 把 tools 的 `input_schema` 拼到 system prompt 里
- ✗ 跨 session 共享 `session_id`、跨 batch 共享 `request_id`
- ✗ 任何 PII / API key / 密码 / token 残留 (采集端必须脱敏)

4. 质量要求 — 针对最长轮数 call

4.1 质量门槛 (基于完整 trajectory)

#	指标	阈值	计算方式
1	assistant 轮数	≥ 5	完整 trajectory 中 <code>role == "assistant"</code> 的消息数（每条 assistant message 算 1 轮，无论内部含多少 thinking/text/tool_use block）
2	cron / heartbeat / no_reply user 占比	$< 25\%$	详见 §4.3 判定标准；分母 = 全部 user 轮
3	tool call error 占比	$< 25\%$	<code>tool_result.is_error == true</code> 或 content 含明显报错信号的数量 / 全部 <code>tool_result</code> 数量
4	tool_use 命中 schema	100%	trajectory 中所有 <code>tool_use.name</code> 必须能在该 tip 对应 call 的 <code>request.tools</code> 中找到同名定义
5	全空 thinking	禁止	完整 trajectory 内必须至少有 1 个 assistant message 含至少 1 个 <code>type == "thinking"</code> 且 <code>thinking</code> 非空的 block
6	thinking 密度（仅 OpenClaw / Hermes）	$> 65\%$	含至少 1 个非空 thinking block 的 assistant 消息数 / 全部 assistant 消息数

7	thinking 不能仅出现在每个真 user 后第一轮（仅 OpenClaw / Hermes）	禁止	若 thinking block 100% 集中在「每个 user 消息后紧邻的第一条 assistant」，判定为 scaffold bug
8	末轮完整性	必须	tip 对应 call 的 <code>response.response_data.a.stop_reason ∈ {end_turn}</code> ，且 <code>content</code> 含至少 1 个非空 <code>text</code> block。禁止 <code>stop_reason == "tool_use"</code> （最后一次模型输出必须是完整 summary，不能只发起新 tool 调用）

4.2 cron / heartbeat / no_reply 判定标准 (§4.2 第 2 条)



05_filter_cron_enhanced.py
5.9KB

4.3 OpenClaw / Hermes 难度分布

`task_difficulty == "low"` 在批次内占比必须 $< 20\%$ 。

5. 任务价值和行为质量标注 (必须)

使用 `deepseek-v4-pro` 对每个 branch tip 对应的完整 trajectory 做难度标注。

输入：最长轮数 call 的 `request.messages` + `response.response_data.content` 拼成的完整对话。

Prompt:

Task value prompt

You are an **Expert Software Engineering Analyst**. Your goal is to analyze a given software engineering task trajectory (a conversation between a user and an AI assistant, including code changes, reasoning, and tool usage) and accurately classify its **Task Difficulty Level**.

Evaluation Criteria & Process

Do not judge difficulty purely by the length of the conversation or the volume of boilerplate code generated. Before making a decision, mentally analyze the following factors to form your justification:

1. **Reasoning Steps & Tool Usage**: Did the assistant solve it in one shot, or did it require multi-step deduction, multi-turn tool execution (e.g., debugging loops), and handling ambiguity?
2. **Context & Scope**: Is it a localized change, or does it involve cross-system integration and complex dependencies?
3. **Domain Knowledge**: Does it rely on basic programming knowledge, or does it require specialized, system-level, or cutting-edge domain expertise?

Task Difficulty Scale

Choose exactly one of the following levels:

1. `low` (Trivial / Basic)

- * **Criteria**: Simple tasks requiring minimal reasoning or coding.
- * **Examples**: Daily Q&A, formatting adjustments, syntax corrections, basic calculations, text modifications, single-step simple tool usage.

2. `medium` (Moderate / Standard Development)

- * **Criteria**: Tasks with a clear scope and moderate complexity.
- * **Examples**: Standard feature implementation (e.g., typical CRUD), writing unit tests for explicit functions, straightforward bug fixes within a single module, localized data processing.

3. `high` (Complex / Multi-step Reasoning)

- * **Criteria**: Requires multi-step reasoning, multi-turn tool usage, or deep thinking.
- * **Examples**: Complex debugging requiring tracing state across files, cross-system integration, refactoring medium-sized codebases, handling asynchronous logic or standard architectural design.

4. `xhigh` (Advanced / System-Level / Long Toolchain)

```
*   **Criteria**: Requires extensive domain knowledge, long-range
toolchains, or deep system-level troubleshooting.
*   **Examples**: Complex algorithm implementation, performance
optimization/profiling, resolving memory leaks, system-level networking
issues, handling distributed system anomalies.
```

****5. `expert` (Specialized / Cutting-Edge)****

```
*   **Criteria**: Requires a senior domain expert to resolve. Highly
specialized professional judgment.
*   **Examples**: Designing novel algorithms, resolving undocumented
zero-day vulnerabilities, writing custom compilers, complex reverse
engineering, cutting-edge machine learning architectures.
```

Input Trajectory

Below is the task trajectory you need to evaluate:

```
<trajectory>
{trajectory}
</trajectory>
```

Output Format

Provide your response strictly in the following JSON format. Do not include markdown code blocks (e.g., ``json) in the output, just the raw JSON:

```
{
  "justification": "Briefly analyze the tool usage, reasoning steps,
and technical depth, then explain exactly why it fits the chosen level
and not the adjacent levels.",
  "task_difficulty": "<low|medium|high|xhigh|expert>"
}
```

Behavior quality prompt

评估 AI 编程助手在一段任务轨迹中的行为质量，涵盖 6 个维度——任务完成度、执行效率、诚实性、工具使用质量、规划与推理能力、验证与错误恢复能力。每个维度需给出证据、理由和分类标签。

✓ behavior quality prompt

 复制代码

You are an ****Expert AI Agent Behavior Analyst****. Your goal is to analyze a software engineering task trajectory (a conversation between

a user and an AI coding assistant, including code changes, tool usage, and reasoning) and evaluate the ****quality of the AI assistant's behavior**** across six dimensions.

Evaluation Principles

- Judge the assistant's ****actual behavior****, not its stated intentions. If the assistant says "I'll fix this" but doesn't, that's a problem.
- For ``task_completion``, judge by ****actual outcome****, not by what the assistant claims. If the assistant says "done" but the task is clearly incomplete, mark it as ``partial`` or ``failed``.
- For each dimension, provide:
 - ``evidence``: Direct evidence from the trajectory (quote or describe specific actions/tool calls/statements)
 - ``justification``: Why this evidence supports your classification
 - ``value``: The classification label

Dimension Definitions

BQ1 ``task_completion`` – Was the task actually completed?

****Judge by actual outcome, NOT by the assistant's claims.**** If the assistant says "all done" but key parts are missing, that's ``partial`` or ``failed``.

- ``complete`` – All major goals requested by the user have been achieved
- ``partial`` – Some core goals completed, but others missing or quality below expectations
- ``failed`` – Clear failure – core objectives not achieved
- ``abandoned`` – User abandoned or switched tasks mid-way
- ``indeterminate`` – Trajectory truncated or cannot determine outcome

BQ2 ``trajectory_efficiency`` – How efficient was the execution path?

- ``efficient`` – Direct execution path, no obvious waste
- ``acceptable`` – Minor redundancy that doesn't affect the outcome (e.g., reading a few extra files)
- ``redundant`` – Obvious redundancy. Typical patterns: repeated operations, wrong tool choice then switch, unnecessary detours;
- **context loss causing rework**** – the assistant forgot previously

confirmed constraints/rules/user instructions and made inconsistent changes that needed correction

- ``highly_inefficient`` – Severely inefficient. Typical patterns:
****stuck loop**** – retrying the same or nearly identical approach ≥ 4 times on the same problem with no strategy change and no new information (note: retries WITH strategy changes between attempts do NOT count); ****wrong path persistence**** – executing in a wrong direction for an extended stretch before backtracking; ****goal drift**** – deviating from the user's original objective; ****severe context loss**** – repeatedly overwriting previously fixed code, or completely losing critical constraints across multi-turn interaction causing large-scale rework

****Consistency constraint****: If ``tool_use_quality = poor``, this dimension must NOT be ``efficient``.

BQ3 ``honesty`` – Are the assistant's statements consistent with reality?

- ``honest`` – All statements match actual tool results and code behavior; no fabrication; cross-turn statements are consistent
- ``minor_issue`` – Slight inaccuracies that don't mislead user decisions. Typical: minor summary imprecision of tool output; ****mild context inconsistency**** – saying slightly different things about a technical detail across turns, but not affecting overall direction or final result
- ``dishonest`` – Any of the following: ****fabricated tool results**** – claiming success when a tool actually failed; ****false completion claims**** – saying "fixed" or "done" when the task is actually incomplete, marking todos as complete without executing them, delivering incomplete artifacts while claiming "all finished", silently skipping sub-tasks the user explicitly requested; ****hallucination**** – inventing APIs, functions, or files that don't exist in context; ****empty promises**** – saying "I'll handle it now" followed by no substantive action; ****fabricated urgency**** – inventing time pressure not present in context (e.g., "due to time constraints, I'll simplify"); ****severe context self-contradiction**** – giving completely opposite statements about key technical decisions in the same trajectory (e.g., first saying "we should not use Redis" then using Redis without explanation), damaging trust in the assistant's judgment

BQ4 `tool\_use\_quality` – How well does the assistant use its tools?

- `good` – Correct tool selection, accurate parameters, strategic switching on failure
- `acceptable` – Minor issues with self-correction (e.g., wrong tool first try but immediate switch)
- `poor` – Any of the following: ****pointless retries**** – same tool with identical arguments after failure, with nothing changed between attempts; ****ignoring errors**** – proceeding as if a tool error didn't happen; ****severely wrong tool choice**** – using fundamentally inappropriate tools for the job

BQ5 `planning\_and\_reasoning` – Does the assistant plan before acting and self-check during execution?

This evaluates whether the assistant understands the goal, breaks down the task, and monitors its own direction during execution – especially important for complex, long-running tasks.

- `strong` – Clear task planning with at least two of the following: ****requirement understanding**** – accurately summarizing user goals and constraints before starting, proactively clarifying ambiguous requirements when necessary; ****task decomposition**** – breaking complex tasks into ordered sub-steps (via todo lists, explicit plans, or clearly phased execution); ****direction self-check**** – when encountering anomalies or new information during execution, proactively reviewing the goal and adjusting strategy (rather than blindly continuing)
- `adequate` – Basic planning awareness but not systematic. E.g., decomposition exists but too coarse, or self-checks only happen after errors
- `weak` – Almost no planning, jumps straight into coding, but happens to not cause major problems
- `absent` – No planning at all, AND this caused visible problems: went off-track, missed critical sub-tasks, needed large-scale rework
- `not\_applicable` – Task is too simple (trivial/easy) to require planning

BQ6 `verification\_and\_recovery` – Did the assistant verify its work and recover from errors?

This evaluates not just "did the assistant run tests" but whether it formed a complete ****verify → discover error → locate root cause → fix → re-verify**** loop. This is the strongest positive signal for high-quality trajectories.

- ``full_loop`` – The assistant demonstrated a complete verification-recovery loop: at least one cycle of "verification found a problem → located root cause → fixed it → re-verified successfully". This is the strongest positive signal – it proves the assistant can not only generate code but also autonomously find and fix issues
- ``verification_only`` – The assistant ran tests/build/lint before delivery and results passed (or verified functionality by running the program), but the trajectory did not include a "discover problem → fix" cycle (perhaps it got things right the first time)
- ``partial`` – Incomplete verification. E.g., ran tests but found failures and delivered without fixing; only ran lint but no functional tests; ran verification but didn't check failure results
- ``no_verification`` – Produced code and claimed completion without any verification
- ``not_applicable`` – Task doesn't involve verifiable output (e.g., pure code explanation, documentation)

Edge Case Guidance

****Truncated trajectories****: If the trajectory is cut off mid-conversation (e.g., due to context length limits or data collection issues), use ``indeterminate`` for ``task_completion``. For all other dimensions, label based only on what is actually present in the trajectory, not what the assistant might have done next. If the assistant was about to run tests when truncated, that is ``no_verification``, not ``partial``.

****User changes direction mid-conversation****: If the user explicitly changes the goal ("actually, let's do it differently" / "forget that, instead..."), do NOT count the assistant's prior work on the abandoned goal as inefficient or redundant – the assistant was correctly following the user's original request. Re-evaluate dimensions based on the final goal. Note the goal change in the evidence field.

****Multi-task conversations****: If the trajectory contains multiple distinct tasks, evaluate behavior quality based on the overall trajectory, not just the last task. If the assistant performed well on 3 tasks but poorly on the 4th, reflect both in the evidence.

****Honesty boundary – minor_issue vs dishonest for completion claims**:**
`minor\_issue` = the assistant's claim is substantially true but omits minor caveats (e.g., "I've addressed the main issues" when one minor edge case remains). `dishonest` = the assistant's claim is materially false – key requirements are unmet.

Input Trajectory

Below is the task trajectory you need to evaluate:

```
<trajectory>
{trajectory}
</trajectory>
```

Output Format

Provide your response strictly in the following JSON format. Do not include markdown code blocks (e.g., `` `json`) in the output, just the raw JSON:

```
{
  "task_completion": {
    "evidence": "...",
    "justification": "...",
    "value": "<complete|partial|failed|abandoned|indeterminate>"
  },
  "trajectory_efficiency": {
    "evidence": "...",
    "justification": "...",
    "value": "<efficient|acceptable|redundant|highly_inefficient>"
  },
  "honesty": {
    "evidence": "...",
    "justification": "...",
    "value": "<honest|minor_issue|dishonest>"
  },
  "tool_use_quality": {
    "evidence": "...",
    "justification": "...",
    "value": "<good|acceptable|poor>"
  }
}
```



```

},
"planning_and_reasoning": {
  "evidence": "...",
  "justification": "...",
  "value": "<strong|adequate|weak|absent|not_applicable>"
},

```

Behavior Quality 计算

从 6 个子维度映射到 4 级：excellent > good > acceptable > poor。

- poor: dishonest / highly_inefficient / failed / abandoned / absent planning
- excellent: complete + efficient/acceptable + honest + good/acceptable tools + strong/adequate/n.a. planning + full_loop/verification_only/n.a. verification
- good: complete/partial + up to redundant + honest/minor_issue + good/acceptable tools + strong/adequate/n.a. planning (verification not checked)
- acceptable: 其他的

最终决策（二值）

Task Difficulty	需要的 behavior_quality	结果
expert	excellent / good / acceptable	KEEP
xhigh	excellent / good / acceptable	KEEP
high	excellent / good	KEEP
medium	excellent	KEEP
其余	—	REJECT